

Smart Elderly Activity Monitoring & Fall Detection System

Wearable IMU • AI-Based Activity Recognition • Cloud-Assisted MVP • Caregiver Dashboard

1. Problem Statement

Senior citizens living alone or in assisted-living environments may require continuous monitoring of their daily activities and rapid assistance after a fall. Conventional monitoring approaches can depend on cameras or cloud processing, creating privacy, connectivity and latency concerns. The proposed system is a low-cost wearable solution that uses an IMU sensor and AI to recognize daily activities, detect potential falls and provide caregivers with a dashboard for activity monitoring and safety alerts.

2. Proposed Solution

The MVP will use a small wearable device containing an MPU6050 accelerometer/gyroscope and an ESP32. The ESP32 collects motion data and sends it over Wi-Fi to a Python-based server. The server performs preprocessing and AI inference using lightweight activity-recognition and fall-detection models trained primarily on existing public datasets. The resulting activity information and safety events are displayed on a caregiver dashboard.

Wearable sensing	MPU6050 captures 3-axis acceleration and 3-axis angular velocity.
Sensor node	ESP32 reads the IMU and transmits sensor windows over Wi-Fi.
Server/AI	Preprocessing + trained activity and fall-detection models run on a normal CPU/server.
Data layer	Activity states, durations and fall events are timestamped and stored.
Dashboard	Caregiver views current activity, daily activity timeline, summaries and fall events.
Alert	A detected fall generates a high-priority alert and is logged for later review.

3. System Architecture

MPU6050 → ESP32 → Wi-Fi → Python Server → Preprocessing → Activity/Fall Models → Database/Event Logger → Caregiver Dashboard

The MVP deliberately separates sensing from AI inference. The wearable remains simple and low-cost, while the computationally heavier machine-learning operations are performed by the server. This avoids the hardware and software complexity of NPU deployment while allowing the complete application workflow to be demonstrated within the project timeline.

4. Detailed Implementation Methodology

Phase 1 — Dataset Selection & Understanding

- Use an existing public IMU-based Human Activity Recognition dataset for the activity-recognition component.
- Use a dedicated public fall-detection/ADL dataset for the fall-detection component rather than assuming that a general HAR dataset contains fall labels.
- Inspect sensor axes, sampling frequency, labels, subject IDs and class distribution.
- Define a common sensor-data format so that the model input produced from the live MPU6050 is compatible with the training pipeline.

Phase 2 — Data Preprocessing

- Clean and normalize accelerometer and gyroscope channels.
- Divide continuous sensor streams into fixed-length time windows.
- Generate the same preprocessing representation for both training data and live ESP32 data.

- Use subject-separated train/validation/test splits where the dataset provides subject information to reduce data leakage.
- Handle class imbalance if required using appropriate sampling or class weights.

Phase 3 — Activity Recognition Model

- Begin with a simple baseline classifier to establish a reference.
- Train a compact model such as a small MLP or 1D-CNN on the IMU windows.
- Evaluate overall accuracy, per-class precision/recall and the confusion matrix.
- Select a model based on both recognition performance and computational simplicity.

Phase 4 — Fall Detection Model

- Build a separate fall-detection classifier using a compatible public dataset.
- Initially target a simple **fall vs normal activity** classification problem.
- Evaluate precision, recall, F1-score and false-alarm behavior because fall detection is safety-sensitive.
- Use a short temporal confirmation rule after a high-confidence fall prediction if required to reduce accidental alerts.

Phase 5 — Server Inference API

- Create a lightweight Python API using a framework such as FastAPI or Flask.
- Receive timestamped IMU windows from the ESP32.
- Run the same preprocessing pipeline used during training.
- Pass each window through the activity and fall models.
- Return compact results such as activity label, confidence, fall status and timestamp.

Phase 6 — ESP32 + IMU Integration

- Connect MPU6050 to ESP32 using I²C.
- Sample acceleration and gyroscope readings at the required rate.
- Buffer measurements into windows matching the ML input format.
- Transmit sensor windows to the server over Wi-Fi.
- Optionally drive a local vibration motor/buzzer when a confirmed fall event is received.

Phase 7 — Dashboard & Data Logging

- Create a lightweight web dashboard connected to the server/database.
- Display the user's current activity.
- Aggregate consecutive predictions into activity durations.
- Display a daily activity timeline and summary.
- Display fall events with timestamp and alert status.
- Keep the dashboard focused on monitoring rather than performing the AI inference itself.

Phase 8 — Integration & Testing

- Test individual components before combining the complete pipeline.
- Measure sensor-to-server transmission delay, model inference time and end-to-end alert latency.
- Test normal activities and controlled fall-like events.
- Analyze false positives and false negatives.
- Prepare a repeatable final demonstration and document system limitations.

5. Machine Learning Pipeline

Existing public datasets

The project will primarily rely on publicly available datasets so that the team does not spend the majority of the 15–20 day period collecting and labeling sensor data. The UCI Human Activity Recognition dataset is a suitable starting point because it contains accelerometer and gyroscope time-series data and six common activities. A separate fall-oriented dataset will be selected for the safety component.

Training pipeline

Dataset → cleaning → normalization → windowing → train/validation split → model training → evaluation → model export

Live inference pipeline

MPU6050 → ESP32 → Wi-Fi → server → same preprocessing → trained model → activity/fall result

6. Activity Recognition Output

Activity	Dashboard interpretation
Walking	Active movement
Walking upstairs	High-intensity movement
Walking downstairs	High-intensity movement
Sitting	Sedentary state
Standing	Stationary upright state
Lying	Resting state

7. Fall Detection & Alert Workflow

Fall detection is treated as a separate safety-critical task. A potential fall is identified from a temporal window of IMU measurements. The system can apply a short confirmation stage to reduce false alarms before generating the caregiver alert.

IMU event → fall model → high-confidence fall → optional confirmation → alert + timestamped event → dashboard

8. Dashboard Design

Module	Information shown
Current activity	Latest recognized activity and confidence.
Activity duration	Approximate time spent in each recognized state.
Daily timeline	Chronological activity history for the day.
Activity summary	Compact view of overall daily activity.
Fall events	Timestamp, event status and alert history.
System status	Connection/device status for the prototype.

9. Evaluation Metrics

Metric	Purpose
Activity accuracy + confusion matrix	Measure recognition quality and class confusion.
Fall precision / recall / F1	Measure detection quality and false-alarm behavior.

Metric	Purpose
Model inference time	Measure AI processing time on the server.
End-to-end alert latency	Measure time from sensor event to caregiver alert.
Data transmission rate	Estimate network requirement of the prototype.
System reliability	Measure successful sensor-to-dashboard operation during testing.

10. MVP Hardware & Expected Cost

Component	Qty.	Indicative cost
ESP32 development board	1	■400–■700
MPU6050 IMU	1	■100–■250
Li-ion/LiPo battery	1	■200–■400
TP4056 charging/protection module	1	■50–■100
Vibration motor + simple driver	1	■50–■130
Breadboard / jumper wires / connectors	—	■100–■200
Enclosure / wearable strap	—	■150–■300
Miscellaneous / contingency	—	■200–■300
Estimated MVP total	—	■1,250–■2,380

No NPU/edge-AI development board is required for the MVP. The AI models will run on the server/laptop. This keeps the hardware cost low and avoids platform-specific NPU deployment work during the 15–20 day development period.

11. 15–20 Day Development Plan

Days 1–2	Finalize datasets, activity/fall labels, hardware and software architecture.
Days 3–5	Preprocess public datasets and build the common sensor-window pipeline.
Days 6–8	Train/evaluate activity-recognition models and select a compact model.
Days 9–10	Train/evaluate fall-detection model and tune alert/confirmation logic.
Days 11–12	Build Python inference API and test model inference using simulated sensor data.
Days 13–14	Integrate MPU6050 + ESP32 and transmit live sensor windows.
Days 15–16	Build dashboard, activity aggregation and event logging.
Days 17–18	Integrate fall alerts and test the complete pipeline.
Days 19–20	Performance testing, documentation, final demo and presentation.

12. Future Scope — Edge AI / NPU Deployment

- Move the trained activity and fall-detection models from the server to an edge-AI development board.
- Apply model quantization and hardware-specific optimization for NPU inference.
- Compare cloud/CPU inference against edge/NPU inference in terms of latency, bandwidth, privacy and power.
- Enable operation during loss of internet connectivity.
- Reduce transmission of raw IMU data by sending only compact activity/event information.

- Evaluate a fully self-contained wearable/edge architecture as a future version.

13. Final MVP Demonstration

- User wears the IMU device and performs normal daily activities.
- ESP32 transmits sensor windows to the server.
- The AI system recognizes the activity and updates the dashboard.
- The dashboard builds a daily activity timeline and duration summary.
- A controlled fall-like event is introduced for demonstration.
- The fall model identifies the event and triggers a high-priority dashboard alert.
- The event is stored with its timestamp for later review.

14. Scope Boundaries

- **Included:** single-user wearable prototype, IMU sensing, existing public datasets, activity recognition, fall detection, cloud/server inference, dashboard, event logging and performance evaluation.
- **Excluded:** medical diagnosis, clinical-grade validation, multi-person tracking, camera surveillance, ambulance/emergency-service integration and large-scale deployment.
- **Safety:** the prototype is intended for academic demonstration and is not a clinically validated emergency-response device.

Project positioning: The MVP validates a complete low-cost elderly monitoring pipeline using wearable motion sensing and AI. The future NPU version can move inference from the server to the edge, making the system more autonomous, private and resilient to network outages.